

MongoDB Cluster Creation and Operations

MongoDB Atlas, CRUD Operations, Indexing and Aggregation Framework

1. MongoDB Cluster Creation

1. Launch your IDE / MongoDB Atlas.
2. Sign up using your Organization credentials.
3. Create a Free Tier cluster.
4. If the cluster is already created, click RESUME.
5. Click on Browse Collections.
6. Go to Atlas Dashboard → Database → Browse Collections → Add My Own Data.
7. You are now inside MongoDB Atlas Data Explorer, where databases and collections can be viewed and managed.

2. CRUD Operations Using MongoDB Atlas UI

CRUD stands for Create, Read, Update and Delete. These operations can be performed directly using the MongoDB Atlas Data Explorer.

a) Create (Insert)

Click INSERT DOCUMENT in the top-right corner.

A JSON editor will open. Enter the following document:

```
{
  "name": "Alice",
  "email": "alice@example.com",
  "movie_id": "12345",
  "text": "This is my first review!",
  "date": "2025-08-23T10:00:00Z"
}
```

Click Insert to save the document.

b) Read (Find)

In the Filter box, type the query in JSON format.

Example: Find comments by Alice:

```
{ "name": "Alice" }
```

Click Apply.

c) Update

Find the document first using a filter such as { "name": "Alice" }.

Hover over the document and click the pencil icon (Edit).

Modify the required field, for example change "text" to "Updated review!".

Save the document.

d) Delete

Hover over the document you want to remove.

Click the trash bin icon (Delete).

Confirm the deletion.

This is how CRUD operations can be performed using the MongoDB Atlas UI.

3. Creating a Database and Collection in Atlas

1. Go to your Cluster → Collections.
2. Click + Create Database.
3. Enter the Database Name, for example: schoolDB.
4. Enter the Collection Name, for example: students.
5. Click Create.

To run insert, update and delete queries, toggle New Data Explorer (the switch beside PREVIEW). Inside the Data Explorer, an editor similar to MongoDB Compass is available for running queries.

4. Using MongoDB Shell (mongosh)

Step 1: Install mongosh

Download MongoDB Shell from:

<https://www.mongodb.com/try/download/shell>

Install it on your computer.

Step 2: Connect to Atlas

In MongoDB Atlas, go to Connect → Connect with MongoDB Shell.

Copy the connection command provided by Atlas.

Example: `mongosh "mongodb+srv://cluster0.psz6ygr.mongodb.net/" --apiVersion 1 --username ramyakrishna`

Open Command Prompt and change the directory to the location where MongoDB Shell is installed.

Paste the command and press Enter.

Enter the Atlas password when prompted.

Output:

```
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>cd C:\Users\rohit\Downloads\mongosh-2.9.2-win32-x64\mongosh-2.9.2-win32-x64\bin

C:\Users\rohit\Downloads\mongosh-2.9.2-win32-x64\mongosh-2.9.2-win32-x64\bin>mongosh "mongodb+srv://"
The system cannot find the file specified.

C:\Users\rohit\Downloads\mongosh-2.9.2-win32-x64\mongosh-2.9.2-win32-x64\bin>mongosh "mongodb+srv://"
The syntax of the command is incorrect.

C:\Users\rohit\Downloads\mongosh-2.9.2-win32-x64\mongosh-2.9.2-win32-x64\bin>mongosh "mongodb+srv://"
Enter password: *****
Current Mongosh Log ID: 6a8934df4dd667d35e73abfb
Connecting to:      mongodb+srv://<credentials>@cluster0.jlsufmt.mongodb.net/?appName=mongosh+2.
Using MongoDB:      8.0.29 (API Version 1)
Using Mongosh:       2.9.2
Mongosh 2.10.0 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/
```

Step 3: Run Queries

Select the database:

```
use schoolDB
```

Insert one document:

```
db.students.insertOne({ name: "Alice", age: 21, major: "Computer Science" })
```

Find a document:

```
db.students.find({ major: "Computer Science" })
```

Output:

```
Atlas atlas-j710rs-shard-0 [primary] test> use schoolDB
switched to db schoolDB
Atlas atlas-j710rs-shard-0 [primary] schoolDB> db.students.insertOne({ name: "Alice", age: 21, major: "Computer Science" })
{
  acknowledged: true,
  insertedId: ObjectId('6a8935634dd667d35e73abfc')
}
Atlas atlas-j710rs-shard-0 [primary] schoolDB> db.students.find(
|   major: "Computer Science"
| )
[
  {
    _id: ObjectId('6a8935634dd667d35e73abfc'),
    name: 'Alice',
    age: 21,
    major: 'Computer Science'
  }
]
```

```
db.students.updateOne({ name: "Alice" }, { $set: { age: 22 } })
```

Output:

```
Atlas atlas-j710rs-shard-0 [primary] schoolDB> db.students.updateOne(
|   { name: "Alice" },
|   { $set: { age: 22 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
db.students.deleteOne({ name: "Alice" })
```

Output:

```
Atlas atlas-j710rs-shard-0 [primary] schoolDB> db.students.deleteOne({ name: "Alice" })
{ acknowledged: true, deletedCount: 1 }
```

5. Indexing in MongoDB

Indexes make searches much faster, similar to an index in a book.

Step 1: Insert Multiple Documents

```
db.students.insertMany([
  { name: "Alice", age: 21, major: "Computer Science" },
  { name: "Bob", age: 22, major: "Mathematics" },
  { name: "Charlie", age: 23, major: "Physics" },
  { name: "David", age: 21, major: "Computer Science" },
])
```

```
    { name: "Eve", age: 22, major: "Mathematics" }
  ]);
```

Output:

```
Atlas atlas-j710rs-shard-0 [primary] schoolDB> db.students.insertMany([
|   { name: "Alice", age: 21, major: "Computer Science" },
|   { name: "Bob", age: 22, major: "Mathematics" },
|   { name: "Charlie", age: 23, major: "Physics" },
|   { name: "David", age: 21, major: "Computer Science" },
|   { name: "Eve", age: 22, major: "Mathematics" }
| ]);
|
| {
|   acknowledged: true,
|   insertedIds: {
|     '0': ObjectId('6a893b384dd667d35e73abfd'),
|     '1': ObjectId('6a893b384dd667d35e73abfe'),
|     '2': ObjectId('6a893b384dd667d35e73abff'),
|     '3': ObjectId('6a893b384dd667d35e73ac00'),
|     '4': ObjectId('6a893b384dd667d35e73ac01')
|   }
| }
```

Step 2: Create an Index

```
db.students.createIndex({ major: 1 });
```

Here, 1 represents ascending order and -1 represents descending order.

Output:

```
Atlas atlas-j710rs-shard-0 [primary] schoolDB> db.students.createIndex({ major: 1 });
major_1
```

Step 3: View Indexes

```
db.students.getIndexes();
```

Output:

```
Atlas atlas-j710rs-shard-0 [primary] schoolDB> db.students.getIndexes();
[
|   { v: 2, key: { _id: 1 }, name: '_id_' },
|   { v: 2, key: { major: 1 }, name: 'major_1' }
| ]
```

Step 4: Create a Compound Index

```
db.students.createIndex({ major: 1, age: -1 });
```

Output:

```
Atlas atlas-j710rs-shard-0 [primary] schoolDB> db.students.createIndex({ major: 1, age: -1 });
|
| major 1 age -1
```

Step 5: Drop an Index

```
db.students.dropIndex("major_1");
```

Output:

```
Atlas atlas-j710rs-shard-0 [primary] schoolDB> db.students.dropIndex("major_1");
{
  nIndexesWas: 3,
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1787378855, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('HEulzdpsScn+gVirFkjIpt2A60=', 0),
      keyId: Long('7641638953010855938')
    }
  },
  operationTime: Timestamp({ t: 1787378855, i: 1 })
}
```

6. Aggregation Framework in MongoDB

The MongoDB Aggregation Framework is used to process documents and produce computed results, such as counts, averages, sorting and joins.

1. Count Students by Major

```
db.students.aggregate([
  { $group: { _id: "$major", total: { $sum: 1 } } }
]);
```

This groups students by major and counts the number of students in each major.

Output:

```
Atlas atlas-j710rs-shard-0 [primary] schoolDB> db.students.aggregate([
|   { $group: { _id: "$major", total: { $sum: 1 } } }
|   ]);
|
| [
|   { _id: 'Physics', total: 1 },
|   { _id: 'Mathematics', total: 2 },
|   { _id: 'Computer Science', total: 2 }
| ]
```

2. Average Age by Major

```
db.students.aggregate([
  { $group: { _id: "$major", avgAge: { $avg: "$age" } } }
]);
```

Output:

```
Atlas atlas-j710rs-shard-0 [primary] schoolDB> db.students.aggregate([
|   { $group: { _id: "$major", avgAge: { $avg: "$age" } } }
|   ]);
|
| [
|   { _id: 'Physics', avgAge: 23 },
|   { _id: 'Mathematics', avgAge: 22 },
|   { _id: 'Computer Science', avgAge: 21 }
| ]
```

3. Sort Students by Age

```
db.students.aggregate([
  { $sort: { age: -1 } }
]);
```

The value -1 sorts the documents in descending order of age.

Output:

```
Atlas atlas-j710rs-shard-0 [primary] schoolDB> db.students.aggregate([
|   { $sort: { age: -1 } }
|   ]);
|
| [
|   {
|     _id: ObjectId('6a893b384dd667d35e73abff'),
|     name: 'Charlie',
|     age: 23,
|     major: 'Physics'
|   },
|   {
|     id: ObjectId('6a893b384dd667d35e73abfe'),
```

```

    _id: ObjectId('6a893b384dd667d35e73ac01'),
    name: 'Eve',
    age: 22,
    major: 'Mathematics'
  },
  {
    _id: ObjectId('6a893b384dd667d35e73abfd'),
    name: 'Alice',
    age: 21,
    major: 'Computer Science'
  },
  {
    _id: ObjectId('6a893b384dd667d35e73ac00'),
    name: 'David',
    age: 21,
    major: 'Computer Science'
  }
}

```

4. Lookup (Join with Another Collection)

First, create another collection named courses:

```

db.courses.insertMany([
  { course: "DBMS", major: "Computer Science" },
  { course: "Algebra", major: "Mathematics" },
  { course: "Quantum Physics", major: "Physics" }
]);

```

Output:

```

Atlas atlas-j710rs-shard-0 [primary] schoolDB> db.courses.insertMany([
|   { course: "DBMS", major: "Computer Science" },
|   { course: "Algebra", major: "Mathematics" },
|   { course: "Quantum Physics", major: "Physics" }
|   ]]);
|
| {
|   acknowledged: true,
|   insertedIds: {
|     '0': ObjectId('6a893ec64dd667d35e73ac02'),
|     '1': ObjectId('6a893ec64dd667d35e73ac03'),
|     '2': ObjectId('6a893ec64dd667d35e73ac04')
|   }
| }

```

Join the students collection with the courses collection:

```

db.students.aggregate([
  { $lookup: {
    from: "courses",
    localField: "major",
    foreignField: "major",
    as: "enrolledCourses"
  }
}

```

```
    }}  
  ]);
```

Output:

```
Atlas atlas-j710rs-shard-0 [primary] schoolDB> db.students.aggregate([  
  { $lookup: {  
    from: "courses",  
    localField: "major",  
    foreignField: "major",  
    as: "enrolledCourses"  
  }  
  }  
]);  
[  
  {  
    _id: ObjectId('6a893b384dd667d35e73abfd'),  
    name: 'Alice',  
    age: 21,  
    major: 'Computer Science',  
    enrolledCourses: [  
      {  
        _id: ObjectId('6a893ec64dd667d35e73ac02'),
```

The \$lookup operation matches the major field in students with the major field in courses and stores the matching course documents in enrolledCourses.

7. Summary

- MongoDB Atlas provides a cloud-based environment for creating and managing MongoDB clusters.
- CRUD operations allow documents to be inserted, viewed, updated and deleted.
- mongosh allows MongoDB commands to be executed from the command line.
- Indexes improve the speed of data retrieval.
- Compound indexes can be created using multiple fields.
- The Aggregation Framework performs operations such as grouping, counting, averaging and sorting.
- \$lookup can be used to combine related data from two collections.